RoamPredict

Comment fonctionne le code, expliqué pas à pas

Pour l'équipe : ce que fait le programme, quand il change de point d'accès et quand il ne le fait pas, ce que veut dire chaque ligne du journal, et quel fichier fait quoi.

Version du code décrite : commit fdfb6da (branche claude/upbeat-newton-gh5kgn) · 264 tests automatiques · Python 3.11+, bibliothèque standard uniquement · Windows 10/11

Sommaire

1. Le problème et l'idée en une page
2. Comment le lancer, et pourquoi il semble souvent « ne rien faire »
3. Vue d'ensemble : une boucle, quatre fois par seconde
4. Pas à pas dans un tour de boucle
5. Quand RoamPredict bascule, et pourquoi il ne le fait généralement pas
6. Comment un basculement est fait en sécurité
7. Mesurer le résultat : les coupures et la ligne STOP
8. Lire le journal : chaque ligne expliquée
9. Un exemple réel, ligne par ligne
10. Quel fichier fait quoi (les trois parties)
11. Listes et dictionnaires dans le code
12. Règles de sécurité
13. Résultats obtenus et limites honnêtes
14. Glossaire

1. Le problème et l'idée en une page

Quand on marche sur le campus avec un ordinateur portable, le signal Wi-Fi du point d'accès (AP) s'affaiblit. Windows décide seul quand passer à un autre AP. Sur notre campus, il attend trop longtemps : lors de nos marches, **internet ne répondait déjà plus 5 à 12 secondes avant que Windows abandonne l'ancien AP**, puis un basculement Windows prenait entre 0,5 et 10 secondes.

RoamPredict surveille le signal 4 fois par seconde, prévoit que la liaison va lâcher, et demande à Windows de passer plus tôt à un meilleur AP, avant que la connexion ne meure. Ce n'est pas de l'intelligence artificielle : il utilise du filtrage, une tendance en ligne droite, une machine à états et une formule de score, que l'on peut tous expliquer au tableau.

Trois questions font tourner tout le programme :

Question	Comment le code y répond	Partie
La liaison va-t-elle lâcher ?	Nettoyer les mesures, calculer la pente, et laisser une machine à états décider (STABLE, DEGRADING, ROAM_IMMINENT…)	Partie 1 : Prédiction
Vers quel AP aller ?	Noter chaque AP visible du même réseau ; le gagnant doit être nettement plus fort (+8 dB)	Partie 1 : Prédiction
Peut-on basculer en sécurité, et cela a-t-il aidé ?	Porte de sécurité, puis WlanConnect avec le profil enregistré, puis vérification de l'adresse IP ; chaque coupure est chronométrée	Parties 2 et 3

Mots importants : **RSSI** = puissance du signal en dBm, toujours négative ; -50 est excellent, -75 est faible, -85 presque perdu. **SSID** = le nom du réseau (« Campus Connecte »). **BSSID** = l'adresse matérielle d'un AP (bc:a9:93:a2:22:00). Un SSID a beaucoup de BSSID.

2. Comment le lancer

Ouvrez PowerShell dans le dossier du projet (celui qui contient `roampredict`, `README.md` et `pyproject.toml`). Si `python` est introuvable, tapez `py` à la place.

```
cd C:\Users\hp\roampredict-new
git pull                                  # récupérer le code le plus récent
python -m roampredict --version           # doit afficher : code fdfb6da
python -m roampredict doctor              # bilan de santé, ne change rien
python -m roampredict run --probe www.google.com --duration 360            # SANS agir
python -m roampredict run --active --probe www.google.com --duration 360   # AVEC action
python -m roampredict report --days 0.1 --timeline 150                     # résultats
```

Partie de la commande	Signification
run	Lance la boucle de surveillance.
--active	Autorise de vrais basculements. **Sans cette option, le programme ne fait qu'observer** (dry-run) et écrit WOULD_SWITCH au lieu de basculer.
--probe www.google.com	Vérifie aussi internet : une connexion TCP vers www.google.com port 443 toutes les 0,25 s. C'est ce qui mesure les coupures internet.
--duration 360	S'arrête tout seul après 360 secondes (6 minutes).

Partie de la commande	Signification
report --timeline	Liste les coupures et basculements enregistrés dans la base, dans l'ordre chronologique.

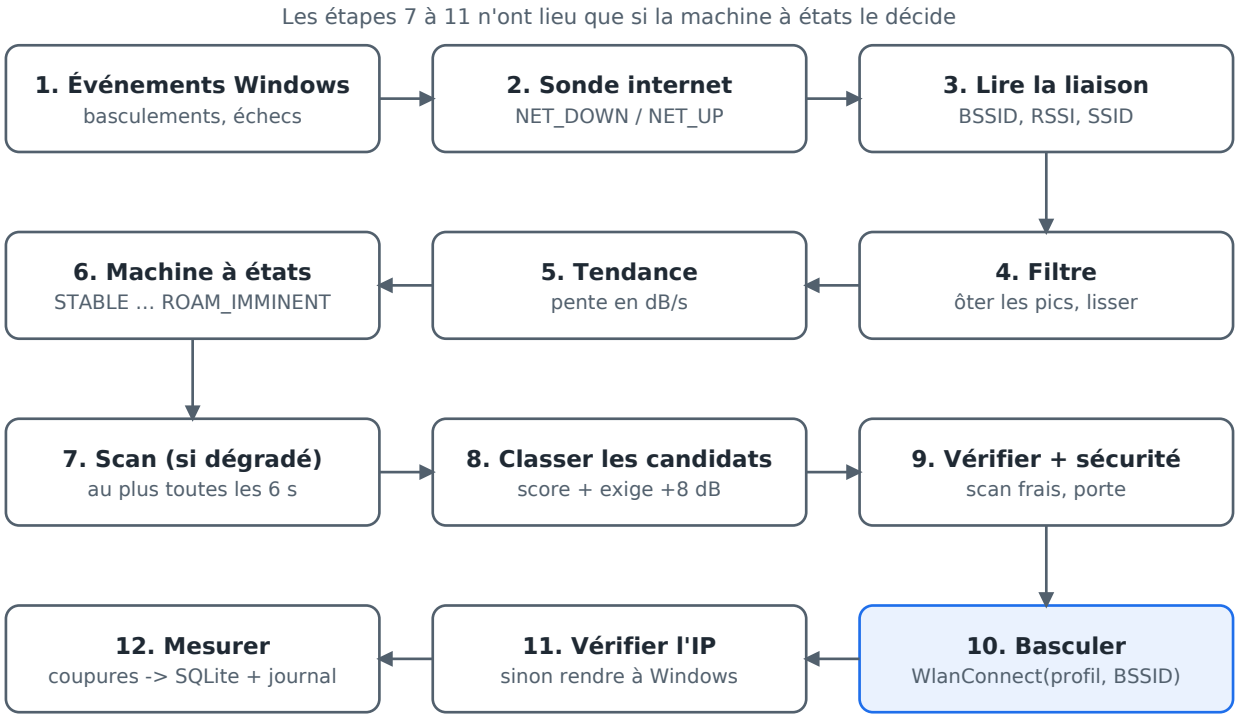
« Ça ne marche pas » : vérifiez d'abord ceci

Ce que vous voyez	Ce que ça veut dire	Que faire
La ligne START dit DRY-RUN	Vous avez oublié --active. Il ne basculera jamais.	Ctrl+C, relancer avec --active.
« Refusing active mode: no SSIDs are allow-listed »	Le fichier de config n'a pas de liste autorisée.	Créer config.toml avec [safety] allowed_ssids = ["Campus Connecte"].
« Refusing active mode: the kill switch is active »	Il s'est arrêté plus tôt (2 basculements ratés ou possible fenêtre de connexion).	Lancer : python -m roampredict unkill
Seulement des lignes STATE, NO_CANDIDATE, WINDOWS_ROAM	Il fonctionne. Aucun AP n'était assez bon, ou Windows a basculé avant.	Normal. Voir la section 5.
« No module named roampredict »	PowerShell n'est pas dans le dossier du projet.	Faire cd dans le dossier.
« unrecognized arguments: --probe »	Ancienne copie du code.	git pull, vérifier --version.

Point clé : RoamPredict n'agit que dans une fenêtre étroite : quand la liaison est prévue comme défaillante (état ROAM_IMMINENT) **et** qu'un autre AP du même réseau est nettement meilleur **et** que toutes les règles de sécurité l'autorisent. La plupart du temps rien de cela n'est vrai, donc il ne fait rien, et c'est correct. Une marche sans aucun basculement n'est pas un plantage.

3. Vue d'ensemble : une boucle, quatre fois par seconde

Le programme est une boucle (`Engine.run` dans `engine.py`). Toutes les 0,25 s, elle appelle `Engine.tick`, qui fait les étapes ci-dessous dans cet ordre. La sonde internet tourne en même temps dans son propre thread.



Les étapes 1 à 6 ont lieu à chaque tour. L'étape 7 seulement en DEGRADING ou ROAM_IMMINENT. Les étapes 8 à 11 seulement en ROAM_IMMINENT. En dry-run, l'étape 10 est remplacée par la ligne WOULD_SWITCH.

Qui appelle qui

Objet (fichier)	Rôle	Utilisé par
Engine (engine.py)	La boucle, toutes les décisions, le chronométrage des coupures	cli.py (commande run)
WindowsWlanAdapter (platform/windows_wlan.py)	Parle à la bibliothèque Wi-Fi de Windows wlanapi.dll via ctypes	Engine, RoamExecutor
SignalFilter, SlidingRegression (signal/)	Nettoie les mesures, calcule la pente	Engine
RoamStateMachine (state_machine.py)	Décide de l'état	Engine
Predictor (topology/predictor.py)	Note les AP candidats	Engine
TopologyStore (topology/store.py)	Mémoire SQLite : AP, déplacements, basculements, coupures	Engine, Predictor, report
SafetyGate (safety.py)	Dry-run, liste autorisée, arrêt d'urgence, limite de fréquence	Engine
RoamExecutor (roam/executor.py)	Réalise un basculement sûr et le mesure	Engine
ConnectivityProbe (net/probe.py)	Vérification internet dans un thread de fond	Engine
Telemetry (telemetry.py)	Lignes à l'écran et fichier journal JSON	tout le monde

4. Pas à pas dans un tour de boucle

Étapes 1-3 : ce qui se passe maintenant

Événements Windows : Windows envoie des notifications (connexion commencée, basculement commencé, déconnexion, échec avec un code). Une fonction de rappel (callback) les range ; le moteur les lit au début de chaque tour. C'est ainsi qu'il voit que **Windows a basculé de lui-même** (ligne WINDOWS_ROAM) ou qu'une connexion a échoué (ligne WARNING avec la raison, par ex. 0x38002 « le pilote s'est déconnecté pendant l'association »).

Sonde internet : toutes les 0,25 s, elle ouvre une connexion TCP vers www.google.com:443 (le nom n'est résolu qu'une fois). **Deux sondes sans réponse d'affilée = une coupure internet** (NET_DOWN). La première réponse la termine (NET_UP, avec la durée). Une seule sonde perdue est ignorée, car des paquets isolés se perdent sur tous les réseaux.

Lire la liaison : `WlanQueryInterface` donne le BSSID, le SSID, le nom du profil et le signal actuels. Sur l'Intel AX201 de notre portable principal, le pilote renvoie une valeur absurde en dBm (+27), donc le code utilise la qualité du signal : `dBm = qualité / 2 - 100`. Si la liaison est perdue, le moteur lance un chronomètre de coupure Wi-Fi et attend que Windows se reconnecte.

Étape 4 : le filtre (signal/filter.py)

Les mesures brutes sautent : une main sur l'antenne peut faire chuter le signal de 10 dB pendant une demi-seconde. Deux étages :

- **Rejet des pics** : une mesure à plus de 6 dB de la médiane des 5 dernières est mise de côté. Elle n'est acceptée que si 2 mesures d'affilée confirment le nouveau niveau. Une mesure étrange ne provoque donc jamais de basculement.
- **Lissage (EMA)** : une moyenne mobile exponentielle avec une constante de temps de 1 seconde : `lissé = lissé + a * (mesure - lissé)`, où a dépend du temps entre deux mesures. Elle enlève le bruit mais ajoute un léger retard.

Étape 5 : la tendance (signal/trend.py)

Le code ajuste une droite (moindres carrés) sur les mesures lissées des **3 dernières secondes**. La pente de la droite est la vitesse de variation en dB par seconde. Elle n'est retenue qu'avec au moins 6 mesures et une tendance nette (la pente vaut au moins 2 fois son incertitude). À partir de la pente, il calcule le **temps avant le seuil** : combien de secondes avant -75 dBm à cette vitesse.

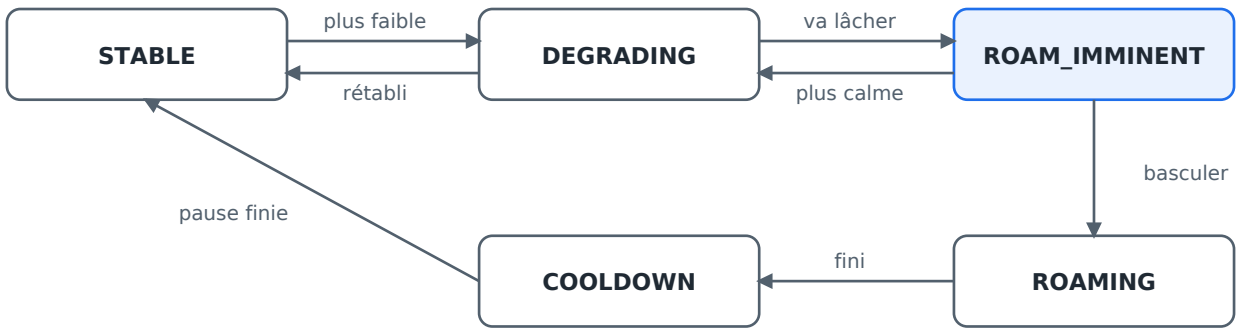
> Exemple : signal à -70 dBm, qui baisse de 3 dB/s : (-70 - (-75)) / 3 = 1,7 seconde restante. Signal à -70 dBm mais stable : « jamais ». Même niveau, danger complètement différent : c'est pour cela que la pente compte.

Étape 6 : la machine à états (state_machine.py)

La machine à états transforme le niveau et la pente en l'un de cinq états. Seuils :

Règle	Valeur
Entrer en DEGRADING	signal sous -67 dBm, ou en baisse avec -75 dBm prévu dans les 10 s
Revenir en STABLE	signal au-dessus de -63 dBm et sans baisse (hystérésis : -67 pour entrer, -63 pour sortir)
Entrer en ROAM_IMMINENT	signal à -75 dBm ou moins, ou baisse d'au moins 1 dB/s avec -75 dBm prévu dans les 4 s, ou (avec --probe) internet ne répond plus alors que le signal est à -65 dBm ou plus faible
Temps de maintien	une condition pire doit durer 0,5 s, une meilleure 2 s (évite le clignotement)
COOLDOWN (pause)	20 s après notre basculement, 60 s après un échec, 5 s après un basculement de Windows

ROAM_IMMINENT est le seul état où RoamPredict peut basculer



STABLE peut aussi passer directement à ROAM_IMMINENT quand le signal s'effondre vite. Quand Windows bascule de lui-même, n'importe quel état passe en COOLDOWN pendant 5 s, puis les mesures repartent de zéro (pendant 9 s la pente est ignorée sur les portables qui donnent une qualité, car la première mesure après une reconnexion est un faux -50).

5. Quand RoamPredict bascule, et pourquoi il ne le fait généralement pas

Un basculement n'a lieu que si **toutes** ces conditions sont vraies, vérifiées dans cet ordre. Chaque « non » a sa propre ligne dans le journal, donc on voit toujours pourquoi rien ne s'est passé.

#	Condition	Sinon : ce que vous voyez
1	L'état est ROAM_IMMINENT	seulement des lignes STATE
2	Pas en COOLDOWN (20 s après un basculement, 60 s après un échec, 5 s après un basculement de Windows)	rien de nouveau
3	Un autre AP du **même SSID** est visible, entendu dans les 30 dernières secondes	NO_CANDIDATE … last heard N s ago
4	Il est au moins **8 dB plus fort** que l'AP actuel (les AP 2,4 GHz comptent 7 dB de moins)	NO_CANDIDATE … only +5.4 dB vs current (needs +8)
5	Ce n'est pas un AP quitté il y a moins de 45 s (sauf s'il est à +12 dB)	… ping-pong guard
6	Si l'AP suivant habituel est visible et que la liaison a encore du temps : l'attendre	WAITING
7	Un scan frais (3 s max) le confirme, et la liaison n'a pas changé entre-temps	NOT_VERIFIED
8	Porte de sécurité : --active, arrêt d'urgence désactivé, SSID autorisé, profil enregistré, limite de fréquence	WOULD_SWITCH … not acting: dry-run

Comment les candidats sont notés (topology/predictor.py)

Chaque candidat reçoit un score entre 0 et 1. Le score classe les candidats ; la règle des +8 dB décide s'ils sont autorisés.

Partie du score	Poids	Signification
appris	0,35	Combien de fois vous êtes passé de cet AP à celui-là auparavant (compteurs dans SQLite)
signal	0,35	Puissance du signal ramenée entre -90 dBm (0) et -45 dBm (1)
vu récemment	0,10	Entendu au dernier scan, s'efface en environ 20 s
connecté récemment	0,20	Vous y étiez connecté dans les 12 dernières heures (sa clé peut être en cache : connexion plus rapide)

Exemple : vous êtes à -72 dBm. Le seul candidat est à -60 dBm en 5 GHz, entendu il y a 2 s, jamais utilisé. Gain = -60 - (-72) = +12 dB, plus que 8, donc autorisé. Score = 0,35 × 1 (appris, c'est le seul) + 0,35 × 0,67 (signal) + 0,10 × 0,90 (vu) + 0,20 × 0 = **environ 0,67**.

La règle de la liaison morte (ajoutée après les tests sur le terrain)

Quand internet est coupé, la mesure du signal de notre portable reste souvent **figée** sur sa dernière valeur : l'AP actuel paraît correct alors qu'il est mort. Avec --probe, si internet ne répond plus et que le signal est à -65 dBm ou plus faible, l'état passe en ROAM_IMMINENT. L'AP actuel est alors jugé d'après ce que le **dernier scan** a mesuré pour lui (ou -85 dBm si le scan ne l'a pas entendu du tout), et l'on prend l'AP **le plus fort** qui le dépasse de **3 dB**. Dans le journal : « internet down; the last scan measured the current AP at -80 dBm; taking the strongest AP (needs +3 dB) ».

6. Comment un basculement est fait en sécurité (roam/executor.py)

Windows n'a pas de commande publique « bascule maintenant ». Le seul outil est `WlanConnect` : « connecte-toi avec ce profil enregistré, et je voudrais ce BSSID ». En général Windows coupe l'ancienne liaison puis se reconnecte, donc un basculement coûte toujours une courte coupure (2 à 7 s mesurées). C'est pourquoi les règles de la section 5 sont strictes.

1. **Vérifier** : relire la liste des AP ; si le candidat n'a pas été entendu dans les 3 dernières secondes, demander un scan et attendre (4,5 s max). Vérifier qu'il est toujours là et toujours assez fort.
2. **Se connecter** : `WlanConnect(nom du profil, un BSSID souhaité)`. Seul le **nom** du profil enregistré est transmis. Le mot de passe ou le certificat reste dans Windows ; Python ne le voit jamais.
3. **Si la connexion échoue** : si l'étape d'authentification 802.1X bloque ou échoue, une fenêtre de connexion pourrait apparaître, donc il s'arrête, le signale et active l'arrêt d'urgence. Sinon il laisse 5 s à Windows pour se reconnecter seul, puis se reconnecte par le seul nom du profil (Windows choisit l'AP).
4. **Vérifier le BSSID** : Windows s'est-il vraiment connecté à l'AP demandé ? Sinon, c'est signalé.
5. **Vérifier la couche IP** : attendre jusqu'à 4 s une vraie adresse et une passerelle. Même réseau : vider le cache ARP ; nouveau réseau : renouveler le DHCP (les deux exigent les droits administrateur, sinon ils sont sautés). Notre campus bloque le ping, donc c'est l'adresse qui est vérifiée au lieu de pinguer la passerelle.
6. **Pas d'adresse IP valable** (vu sur le portable d'Aymane : le DHCP n'a jamais répondu, Windows s'est donné 169.254.x) : **nouveau dans fdfb6da**, il se reconnecte par le seul nom du profil, pour que Windows choisisse à nouveau l'AP et que le portable ne reste pas hors ligne sur l'AP demandé.
7. **Mesurer et retenir** : temps de connexion, d'authentification, d'obtention de l'IP, succès ou échec sont enregistrés dans SQLite. Puis la pause COOLDOWN commence (20 s, ou 60 s après un échec).

Un basculement est « recent », « expired » ou « never » selon que vous étiez connecté à cet AP dans les 12 dernières heures ou non. Le rapport compare leurs durées : si les « recent » sont nettement plus rapides, le campus garde les clés de connexion en cache (PMKSA caching).

7. Mesurer le résultat : les coupures et la ligne STOP

Deux types de coupures sont chronométrés et enregistrés dans la table SQLite `outages`, avec le mode (dry-run ou active) :

- **Coupure Wi-Fi** : du moment où la liaison est perdue jusqu'à une nouvelle connexion (LINK_DOWN … CONNECTED).
- **Coupure internet** : de la première sonde sans réponse jusqu'à la première réponse (NET_DOWN … NET_UP). C'est ce que ressent l'utilisateur, donc c'est le résultat principal.

Pendant l'exécution, elles sont aussi rangées dans un dictionnaire de listes, `session_outages = {"wifi": [], "internet": []}`. À la fin, `session_summary()` les compte, les additionne et prend la plus longue :

```
STOP  RoamPredict stopped (0/0 switches succeeded). This session: Wi-Fi link lost 5x,
      48.5 s in total (longest 21.9 s); internet lost 12x, 162.1 s in total (longest 34.0 s);
      Windows roamed 10x
```

Nos propres basculements comptent aussi comme des coupures, mesurées depuis la demande : RoamPredict ne peut donc paraître que pire, jamais meilleur, que Windows seul. **Comparaison équitable** : même portable, même trajet, même durée, une fois sans --active et une fois avec, puis comparer « internet perdu » en pourcentage de la marche.

8. Lire le journal : chaque ligne expliquée

Les noms de lignes restent en anglais, car c'est ce que le programme affiche.

Ligne	Signification
START	Démarrage ; indique DRY-RUN ou ACTIVE et quelle version du code (commit) tourne.
CONNECTED	Connecté à un AP (au début, ou après une perte : « after 14.5 s without Wi-Fi »).
STATE	La machine à états a changé d'état, avec la raison (rssi, pente, temps avant le seuil).
NO_CANDIDATE	Voulait bouger, mais aucun AP ne convient ; nomme le plus proche et pourquoi il échoue.
WAITING	Attend l'AP suivant habituel, car la liaison a encore quelques secondes.
NOT_VERIFIED	Le meilleur candidat a échoué à la vérification fraîche (pas entendu récemment, ou plus assez fort).
WOULD_SWITCH	Dry-run : tout autorisait un basculement, mais il n'a pas agi.
DECISION	Mode actif : bascule maintenant, avec le gain, le score et la raison.
SWITCH_RESULT	OK ou FAILED, avec les temps de connexion, d'authentification, d'IP et le repli utilisé (fallback).
WINDOWS_ROAM	Windows est passé seul à un autre AP, avec la durée.
PREDICTION_CHECK	Dry-run : Windows a-t-il choisi l'AP que nous aurions choisi ? HIT ou MISS, et combien plus tôt nous avions décidé.
LINK_DOWN	Liaison Wi-Fi perdue, avec les dernières mesures avant la perte.
NET_DOWN / NET_UP	Internet ne répond plus / répond à nouveau, avec la durée et les sondes perdues.

Ligne	Signification
IP_CHANGE	L'adresse IP a changé ; 169.254.x signifie que le DHCP n'a pas répondu, donc pas d'internet.
WARNING	Windows a signalé une tentative de connexion ratée, avec son code et son texte.
KILL_SWITCH	L'arrêt d'urgence est activé ou désactivé.
STOP	Fin de la session avec les totaux (section 7).

Codes d'erreur Windows rencontrés sur le campus

Code	Texte	Ce que ça veut dire
0x38002	Le pilote s'est déconnecté pendant l'association	L'AP n'a pas accepté la connexion (signal faible, AP chargé)
0x38001	L'opération a été annulée	Une tentative de connexion a été interrompue
0x2800b	Le réseau spécifique n'est pas disponible	Aucun AP de ce réseau entendu à ce moment
0x50006	L'authentificateur n'est plus présent	L'AP a disparu pendant l'étape de connexion
0x4800F	Pas d'authentificateur, mais le profil exige 802.1X	L'étape de connexion n'a jamais commencé

9. Un exemple réel, ligne par ligne

Extrait de la marche active d'Aymane (horloge de son portable), légèrement raccourci :

```
16:44:03 NET_DOWN       no answer from www.google.com:443 since 16:44:03.4
16:44:06 STATE          DEGRADING -> ROAM_IMMINENT: internet stopped answering at -68.9 dBm
16:44:10 NOT_VERIFIED   not switching to ...a0:30: no fresh data for the candidate (last heard: 24.1 s)
16:44:14 DECISION       switching ...9a:f0 -> ...24:30 (+27.0 dB, score 0.32, never; internet down
                        and the last scan did not hear the current AP: counted as -85 dBm;
                        taking the strongest AP (needs +3 dB))
16:44:16 SWITCH_RESULT  OK: switched [never, connect 1914 ms (auth 773 ms), IP ready after 13 ms]
16:44:22 NET_UP         internet answers again after 19.0 s
```

1. La sonde rate 2 réponses d'affilée : la coupure internet commence (NET_DOWN).
2. Le signal est à -68,9 dBm (plus faible que -65) et internet est mort : la règle de la liaison morte met l'état en ROAM_IMMINENT.
3. Le premier candidat a été entendu il y a 24 s : trop vieux, donc refusé (NOT_VERIFIED). Un scan frais est lancé.
4. Le scan n'entend plus l'AP actuel, qui compte donc pour -85 dBm. L'AP le plus fort est 27 dB meilleur, bien plus que les 3 dB exigés : DECISION.
5. WlanConnect prend 1,9 s, dont 0,8 s d'authentification 802.1X ; l'adresse IP est prête 13 ms après.
6. 6 s après le basculement, internet répond à nouveau. Coupure internet totale : 19 s.

Dans la même marche, 22 s plus tard, le troisième basculement s'est connecté en 1,6 s mais le DHCP n'a jamais donné d'adresse (IP_CHANGE vers 169.254.x). L'ancien code restait sur cet AP. Depuis fdfb6da, il rend la connexion à Windows. C'est ainsi que les tests sur le terrain améliorent le code.

10. Quel fichier fait quoi (les trois parties)

Partie	Fichiers à maîtriser	Fichiers à survoler
1. Prédiction : quand bouger, et vers où	signal/filter.py (216 lignes), signal/trend.py (158), state_machine.py (252), topology/predictor.py (260)	topology/store.py (570), simulator/ (1 559)
2. Windows et réseau : voir, scanner, se connecter, vérifier internet	platform/base.py (332), platform/windows_wlan.py (649), net/probe.py (177), validate.py (114)	platform/info_elements.py, net/l3.py, net/latency.py, doctor.py
3. Moteur, sécurité, mesures : la boucle, les règles, les résultats	engine.py (936), safety.py (167), roam/executor.py (463)	cli.py, config.py, telemetry.py, buildinfo.py

Tous les chemins sont dans le dossier `roampredict` sauf `simulator/`. Le simulateur (`python -m roampredict simulate --no-plot --seeds 3`, 2 secondes, sur n'importe quel portable) compare Windows seul, un simple seuil et RoamPredict sur un étage d'école virtuel. Les tests : `py -m pip install pytest` une fois, puis `py -m pytest`.

11. Listes et dictionnaires dans le code

Les deux sont utilisés partout. Chaque présentateur peut montrer les exemples de sa partie et expliquer pourquoi cette structure convient.

Part.	Où	Code	Pourquoi cette structure
1	topology/predictor.py:180	pool: dict[str, CandidateInput] = {}	Dictionnaire indexé par BSSID : une entrée par AP, un doublon garde la mesure la plus forte
1	topology/predictor.py:194, :249	results.append(...); results.sort(key=...)	Liste : garde un ordre, les autorisés et mieux notés d'abord
1	topology/store.py:484	groups.setdefault(key, []).append(rec)	Dictionnaire de listes : regroupe les basculements pour calculer des médianes
2	platform/windows_wlan.py:194	PROTOTYPES: dict[str, tuple[list, object]]	Dictionnaire comme table : 11 fonctions Windows préparées par une seule boucle
2	platform/windows_wlan.py:73	_ACM_KINDS = {...}	Table de correspondance : numéro d'événement Windows -> nom, sans long if/elif
2	doctor.py:80	[{k: str(v) for k, v in item.items()} for item in items]	Liste de dictionnaires : compréhension de liste autour d'une compréhension de dictionnaire
3	engine.py:200	session_outages = {"wifi": [], "internet": []}	Dictionnaire de listes : une liste de durées par type de coupure
3	config.py:85	SCHEMA: dict[str, dict[str, type]]	Dictionnaire imbriqué : section -> clé -> type ; les clés inconnues sont refusées
3	cli.py:61, :83	events: list[tuple[float, str]]; events.sort(); events[-last:]	Liste de tuples : trier par heure, garder les N derniers

Autres structures à citer : tuples (paires fixes comme (heure, dBm)), deque (fenêtre glissante de mesures, signal/trend.py:101), frozenset (valeurs autorisées, topology/store.py:120), Counter (raisons des sondes perdues, net/latency.py:114), dataclasses (enregistrements comme LinkStatus), Enum (les cinq états).

12. Règles de sécurité

- **Dry-run par défaut.** Rien ne bascule sans --active.
- **Uniquement son propre appareil et ses propres réseaux autorisés.** Le mode actif refuse de démarrer sans liste autorisée, et n'agit que sur les SSID listés dans config.toml.
- **Aucun mot de passe.** Seul le nom du profil enregistré est transmis à Windows ; le journal supprime tout ce qui ressemble à un identifiant.
- **Pas de fenêtre de connexion.** Si l'étape 802.1X bloque ou échoue, il s'arrête, le signale et laisse tout à Windows (l'arrêt d'urgence s'active).
- **Jamais de déconnexion sans un meilleur AP vérifié** (scan frais, +8 dB, ou +3 dB sur une liaison morte).
- **Limite de fréquence** : au moins 20 s entre deux basculements, au plus 6 par 10 minutes ; arrêt après 2 basculements ratés d'affilée.
- **Arrêt d'urgence** : `python -m roampredict kill` depuis une autre fenêtre bloque tout basculement ; `unkill` le réautorise. Ctrl+C arrête le programme.
- **Chaque BSSID est validé** avant d'atteindre Windows. Pas de désauthentification, d'usurpation ni d'injection de paquets, et pas d'outils réservés à Linux.

13. Résultats obtenus et limites honnêtes

Marche (portable principal sauf mention)	Mode	Internet perdu	Remarque
Atelier, 6 oct., 7:24	sans	230 s (52 %)	11 coupures, sauts vers RobotiCore
Atelier, 6 oct., 5:41	avec	136 s (40 %)	2 basculements : 1 OK (internet revenu 6,6 s après), 1 raté
7 oct., marche 1, 6:00	sans	331 s (92 %)	Une coupure de 257 s : l'authentification de Windows échouait sans cesse
7 oct., marche 2, 6:00	sans	162 s (45 %)	10 basculements Windows, 2 sauts vers RobotiCore
7 oct., Aymane, 6:00	sans	114 s (32 %)	Son portable bascule seul 11 fois
7 oct., Aymane, 2 premières min	avec	93 s sur 129 s (72 %)	3 basculements : 2 OK, 1 sans IP ; batterie vide

Ce que l'on peut dire honnêtement : Windows seul, sur le même portable et le même trajet, a varié de 45 % à 92 % de la marche hors ligne. Les coupures individuelles gérées par RoamPredict ont été courtes (6,6 s et 19 s, contre une médiane d'environ 36 s pour Windows le premier jour), mais il y a trop peu de marches actives pour affirmer une amélioration générale. Les tests sur le terrain ont trouvé et corrigé de vrais problèmes : un pilote donnant des dBm absurdes, une mesure figée quand la liaison est morte, un plantage sur les grands codes d'erreur, et un basculement qui laissait le portable sans adresse IP.

Limites

- Windows n'a pas de commande « bascule maintenant » ; WlanConnect coupe d'abord la liaison, donc chaque basculement coûte quelques secondes.
- Après un basculement par BSSID, certains pilotes peuvent garder le portable sur cet AP (« pinning ») ; `netsh wlan connect name="Campus Connecte"` le libère.
- RoamPredict ne peut pas aider quand le portable est déconnecté ou quand l'authentification de Windows (802.1X) échoue sans cesse.
- Deux portables avec la même carte Wi-Fi se sont comportés très différemment : il faut comparer sur le même portable.
- Peu de marches : les chiffres sont des exemples, pas une preuve. Windows uniquement.

14. Glossaire

Mot	Signification
AP / BSSID	Point d'accès / son adresse matérielle (un AP, un BSSID par bande)
SSID	Nom du réseau, par ex. Campus Connecte
RSSI, dBm	Puissance du signal ; plus c'est proche de 0, plus c'est fort (-50 bon, -75 faible, -85 presque perdu)
Roaming / basculement	Passer d'un AP à un autre du même réseau
Dry-run	Observer et noter seulement, ne jamais basculer
802.1X, WPA2-Entreprise	Connexion avec un compte universitaire ; Windows la fait avec le profil enregistré
DHCP, 169.254.x	Comment un portable obtient son adresse IP / l'adresse que Windows invente quand le DHCP ne répond pas
Sonde (probe)	La vérification internet : connexion TCP vers www.google.com:443 toutes les 0,25 s
EMA	Moyenne mobile exponentielle, le lissage
Pente	Vitesse de variation du signal en dB par seconde, d'après une droite sur les 3 dernières secondes
Hystérésis	Seuils différents pour entrer et sortir d'un état (-67 / -63), pour éviter le clignotement
ctypes	Le moyen intégré à Python d'appeler des fonctions de DLL Windows comme wlanapi.dll
SQLite	Le petit fichier de base de données qui stocke AP, déplacements, basculements et coupures
Arrêt d'urgence (kill switch)	Un fichier qui, tant qu'il existe, bloque tout basculement
Journal (log)	Le fichier roampredict.jsonl où chaque événement est écrit, une ligne JSON par événement
802.11r/k/v	Fonctions de roaming rapide annoncées par les AP du campus ; Windows les utilise pour ses propres basculements